

Incident Report

본 문서는 SRE 미니 프로젝트의 장애 시나리오와 RCA를 정리합니다.

세 가지 시나리오를 검증했습니다.

- case A: Postgres 중지로 인한 사용자 API 5xx 증가 (dependency layer, 메인 RCA)
- case B.1: 애플리케이션 지연 장애 (application layer)
- case B.2: 애플리케이션 5xx 장애 (application layer)

case A: Postgres 중지로 인한 사용자 API 5xx 증가

A.1 장애 요약

항목	내용
Incident	Postgres outage
Trigger	<code>fault-mode=normal</code> 에서 <code>docker stop sre-postgres</code>
Main alert	<code>HighErrorRate</code> (critical)
동반 신호	<code>HighLatencyP95</code> , <code>db_errors_total</code> , <code>/readyz</code> 503
사용자 영향	<code>/api/v1/*</code> 요청이 5xx로 실패
복구	<code>docker start sre-postgres</code> 후 pool reconnect

스크립트는 먼저 `fault-mode=normal` 로 초기화한 뒤 Postgres를 중지합니다.

API 프로세스는 살아 있었지만 사용자 API는 DB connection을 얻지 못해 500을 반환했습니다. `/readyz` 는 503으로 전 환되었지만 Docker Compose에는 이를 트래픽 차단으로 연결하는 routing 계층이 없습니다.

Postgres는 비즈니스 기능을 늘리기 위한 요소가 아닙니다. 의존성 장애가 사용자 SLI로 전파되는 흐름을 검증하기 위한 최소 구성입니다.

A.2 영향 범위

evidence: `evidence/db-stop/02-alert-firing.json`

지표	정상	장애 시점
Error Rate (2m)	0%	100%
Request Rate	200 응답 9.98 rps	500 응답 1.98 rps
Latency p95 (2m)	47.5ms	4.85s
Availability (1h proxy)	100%	73.23%
DB errors	없음	list/get 약 0.49 errors/s

HighErrorRate 가 메인 감지 신호입니다. **HighLatencyP95** 도 함께 firing되었는데, DB connection pool timeout 때문에 실패 응답도 지연되어 반환되었기 때문입니다.

A.3 타임라인

evidence: **evidence/db-stop/timeline.txt**

시각	단계	내용
20:48:41	발생	Postgres 중지
20:51:12	감지	HighErrorRate firing
20:51:12	대응 시작	Postgres 재시작
20:52:47	서비스 복구	/readyz 200, pool reconnect
20:54:33	Alert resolved	rolling 2m window 정상화

지표	값
MTTD	2분 31초
Service MTTR	4분 6초
Alert resolution lag	1분 46초

Alert가 서비스 복구보다 늦게 resolved된 이유는 2분 rolling window에 장애 데이터가 남아 있었기 때문입니다.

A.4 Root Cause

Root Cause: 단일 Postgres 의존성의 unavailability.

Why	Cause
사용자가 왜 5xx를 받았는가?	사용자 API가 DB 조회 실패로 500 반환
DB 조회가 왜 실패했는가?	psycopg pool에서 connection 획득 실패
connection 획득이 왜 실패했는가?	Postgres 컨테이너가 중지됨
API는 왜 트래픽을 계속 받았는가?	Compose에는 readiness 기반 traffic shedding이 없음
왜 영향이 100%였나?	API와 DB가 모두 단일 인스턴스

Root cause는 DB 중지이지만, 사용자 영향으로 번진 이유는 아래 contributing factors가 설명합니다.

A.5 Contributing Factors

Factor	영향
Readiness routing 부재	/readyz 503에도 사용자 요청이 계속 인입
depends_on 의 한계	startup ordering만 보장, 운영 중 장애는 해결하지 못함

Factor	영향
단일 인스턴스	우회 경로가 없어 영향이 100%로 확대
Rolling window	실제 복구 후에도 alert가 잠시 firing 유지

A.6 대응 및 복구

확인:

```
curl -i http://localhost:8000/readyz
docker compose exec postgres pg_isready -U elice -d elice
curl -fsS http://localhost:9090/api/v1/alerts
```

복구:

```
docker start sre-postgres
```

복구 기준:

- `HighErrorRate`, `HighLatencyP95` resolved
- Error Rate < 1%
- Latency p95 < 300ms
- `/readyz` 200
- DB error rate 감소 추세

`evidence/db-stop/04-alert-resolved.json` 기준으로 alert는 resolved되고 p95 latency는 48ms 수준으로 회복되었습니다. `db_errors_5m` 은 5분 window 때문에 잠시 non-zero로 남을 수 있습니다.

운영 환경에서의 차이

본 데모에서 복구는 `docker start sre-postgres` 한 명령이지만, 데이터 정합성 위험과 근본 원인 회피 가능성 때문에 실 운영에서 운영자가 DB 프로세스를 직접 재기동하기는 어렵습니다.

따라서 본 데모의 진짜 검증 가치는 재기동 명령이 아니라:

1. dependency 장애의 사용자 영향 빠른 감지 (MTTD 2:31)
2. layer 식별 능력으로 application/dependency 원인 구분
3. psycopg_pool 자동 reconnect (코드 변경 없이 90초 복구)
4. readyz 503이 사용자 영향 차단으로 이어지지 않는 환경 제약 시연
5. metric 기반 회복 판정 기준 정립

A.7 재발 방지 대책

계층	대책
코드	DB <code>connect_timeout</code> , <code>statement_timeout</code> , pool acquire timeout을 latency budget에 맞춰 명시
코드	retry/backoff 적용 시 retry storm 방지

계층	대책
운영	runbook에 DB 의존성 확인 절차와 복구 기준 유지
플랫폼	/readyz 503이 traffic 차단으로 이어지는 환경에서 운영
플랫폼	DB가 죽어도 자동 복구되는 환경에서 운영
관측	구조화 로그와 trace를 추가해 metric 이후 원인 추적 연결

A.8 잘 동작한 부분

- 사용자 영향 기반 alert가 먼저 동작했습니다.
- DB diagnostic metric이 원인 범위를 빠르게 좁혔습니다.
- /readyz 가 DB 장애를 의도대로 반영했습니다.
- 복구 기준을 metric 기반으로 둔 덕분에 rolling window 지연을 설명할 수 있었습니다.

case B. Application Fault Scenarios

문항 10이 요구하는 API의 세 동작(정상 응답 / 의도적 응답 지연 / 의도적 5xx)을 incident 시나리오와 1:1로 매핑합니다. 메인 RCA(DB stop)는 dependency layer 장애이고, 아래 두 시나리오는 application layer 장애입니다.

B.1 Latency fault (fault-mode slow)

항목	내용
Trigger	POST /admin/fault-mode {"mode":"slow","delay_ms":700}
Alert	HighLatencyP95 (warning)
Error Rate	0% 유지
p95 latency	975ms
DB diagnostic	DB 오류 없음
복구	fault-mode normal

시각	단계	내용
20:56:14	발생	slow 700ms 주입
20:58:45	감지	HighLatencyP95 firing
20:58:45	대응	normal 모드 복귀
21:00:46	복구	alert resolved

B.2 Application 5xx (**fault-mode flaky**)

항목	내용
Trigger	<code>POST /admin/fault-mode {"mode":"flaky","error_rate":0.3}</code>
Alert	<code>HighErrorRate</code> (critical)
Error Rate	29.56% (target 30%에 근접)
p95 latency	47.5ms (정상 유지)
DB diagnostic	DB 오류 series 없음, DB latency 정상
복구	<code>fault-mode normal</code>

시각	단계	내용
23:56:00	발생	flaky 30% 주입
23:58:46	감지	<code>HighErrorRate</code> firing (단독)
23:58:46	대응	normal 모드 복구
00:00:47	복구	alert resolved

B.3 시그니처 비교: 같은 Error Rate alert, 다른 root cause

`HighErrorRate` 는 두 시나리오 모두에서 firing되지만, 동반 신호로 dependency 장애와 application 장애를 구분할 수 있습니다. 같은 paging signal에서 두 layer를 분리하는 것이 DB diagnostic 메트릭 분리 설계의 핵심 가치입니다.

신호	DB stop (dependency)	flaky (application)
HighErrorRate	firing	firing
HighLatencyP95	firing (PoolTimeout)	inactive
error_rate_2m	100%	29.56%
latency_p95_2m	4.85s	47.5ms
<code>db_errors_5m</code>	0.49/s	series 없음
<code>db_latency_p95_5m</code>	timeout	정상
<code>/readyz</code>	503	200
<code>api_fault_mode</code>	normal (DB가 외부 trigger)	flaky=1
운영자 진단 경로	DB 의존성 점검	최근 application 변경 점검

Evidence Map

파일	의미
<code>evidence/db-stop/01-baseline.json</code>	정상 상태

파일	의미
evidence/db-stop/02-alert-firing.json	DB down, HighErrorRate firing
evidence/db-stop/03-service-recovered.json	/readyz 200, alert는 rolling window로 firing 유지
evidence/db-stop/04-alert-resolved.json	alert resolved
evidence/latency/02-alert-firing.json	latency fault, HighLatencyP95 firing
evidence/flaky/01-baseline.json	정상 상태 (flaky 시나리오 baseline)
evidence/flaky/02-alert-firing.json	flaky 30%, HighErrorRate 만 firing, DB 오류 series 없음
evidence/flaky/03-alert-resolved.json	normal 복귀, alert resolved